

SME Session Scheduler

Automates weekly instructor-to-session matching for live cohort learning programs — ingest → match → flag conflicts → human review/approve/override, end to end.

🔗 **Live app:** <https://sme-scheduler-bcapwigvs-hv-716.vercel.app/>

Table of Contents

[What This Is](#)

[Tech Stack](#)

[Getting Started \(Local Setup\)](#)

[How to Use the App — Full Walkthrough](#)

[Project Structure](#)

[Testing](#)

What This Is

Ops/curriculum teams currently spend hours each week manually matching SMEs (instructors) to live sessions in a spreadsheet — a process that's slow and fails silently (a double-booking or expertise mismatch isn't caught until a learner shows up to an unstaffed class).

This app automates that: it ingests a week's sessions and SME pool, applies deterministic hard rules plus a fairness-weighted ranking to auto-draft a schedule, uses targeted LLM reasoning only where real judgment is needed, and puts every assignment in front of a human for approval or override before anything is final.

Tech Stack

Frontend: React + Vite

Backend: Vercel serverless functions (`/api/reason`, `/api/calendar`)

AI reasoning: OpenAI API (`gpt-4o-mini`), called server-side only — the key never touches the browser

Calendar integration: Google Calendar API via OAuth refresh token

CSV parsing: Papa Parse

Getting Started (Local Setup)

```

bash
git clone <this-repo-ur>
cd sme-scheduler
npm install
npm run dev
```

The app will run locally, but the AI reasoning and calendar features need environment variables set (`OPENAI_API_KEY`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `GOOGLE_REFRESH_TOKEN`) — without them, matching still works, the app just shows "AI reasoning layer unavailable" and skips calendar blocking gracefully.

How to Use the App — Full Walkthrough

1. Quick start (30 seconds)

Open the app and click "**▶ Run Matching**" — it uses built-in sample data automatically, no setup needed. Scroll through the draft schedule, check a few "**approve**" boxes, click "**Submit approvals →**".

2. Using your own data

Click " **Data**" in the top bar.

Click "**Download template**" under Sessions (and separately under SME Pool) to see the exact expected CSV format.

Fill in real data in the same format, or edit the template directly.

Click "**Upload CSV**" for each file.

If a file has a problem, a red error banner names the exact row and field — your previously-loaded data stays intact until a valid file replaces it.

Click " **Reset to sample data**" any time to revert to the demo dataset.

3. Reading the draft schedule

Each session shows:

The assigned SME, or "unfilled" if nobody qualified

A confidence score (0-100%) — low confidence usually means a close call between two similarly-good options

A workload dial — green/amber/red circle showing that SME's load relative to the team average this week

A flag, if something's worth attention (unfilled, tie, fairness concern, below-floor fallback)

4. Reviewing flags

The right-hand panel lists every flag by priority — **High** (unfilled sessions), **Medium** (fairness/floor concerns), **Low** (ties, under-utilization). Click any flag to jump straight to the session it concerns.

5. Overriding an assignment

Every session has a dropdown listing everyone hard-rule-qualified. If a pick would exceed someone's cap or cause a double-booking, you'll see `⚠ exceeds cap` or `⚠ would double-book` right in the option — you can still choose them knowingly. If a session has no exact topic match but the AI found a plausible naming-difference match, you'll see a separate `🌀 AI-suggested` group in the dropdown — always a suggestion, never automatic.

6. Handling a last-minute cancellation

Click "**simulate drop-out**" on any assigned session — only that session reopens for reassignment, everything else you've approved stays untouched.

7. Exploring trade-offs

Drag the **Fairness / Performance / Preference** sliders at the top of the draft to see the schedule re-rank live. Click "**↻ Refresh AI notes**" once you've settled on a configuration, to update the AI explanations to match (this doesn't happen automatically on every drag, to avoid burning API calls on every pixel of movement).

8. Approving and finishing up

"Approve all filled sessions" — bulk-approves every assigned session

"↓ Export to Sheets (.csv)" — downloads the schedule in spreadsheet-ready format

"Submit approvals →" — locks the week; approved sessions are automatically pushed to the connected Google Calendar, with a status panel showing which events succeeded

Project Structure

```

sme-scheduler/
├── api/
│   ├── reason.js    # Serverless function: AI reasoning (OpenAI)
│   └── calendar.js   # Serverless function: Google Calendar integration
├── src/
│   ├── SchedulerApp.jsx # The entire app — matching engine, UI, everything
│   └── main.jsx        # React mount point
├── index.html
├── package.json
└── vite.config.js
```

Almost all of the actual logic and UI lives in one file, `src/SchedulerApp.jsx` — the pure matching-engine functions (hard rules, scoring, fairness, timezone conversion, CSV parsing) sit at the top of the file, separate from the React component and JSX below them, which is what makes them independently testable (see below).

Testing

The matching engine, CSV validation, and timezone logic are covered by **1,370+ automated checks** across four independent test suites (targeted scenario tests, a randomized property-based fuzz test, a dedicated fairness-weighting stress test, and a deep-dive suite for regressions/determinism/edge cases) — run directly against the real source code, not a re-typed copy.